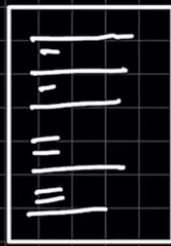


Python a new start:

> python chai.py



Byte Code



mostly hidden



Virtual
Machine

1. Compile to Byte Code

low level & Platform Independent

→ Byte Code runs faster

.pyc → compiled python (frozen Binaries)

→ Byte Code runs faster

.pyc → compiled python (frozen Binaries)

--pycache--

Source Chang & Python Version

hello_chai.cpython-312.pyc

→ Works only for imported files

→ not for top level files

→ Python Virtual Machine PVM

→ Code loop to iterate byte code

→ Run time Engine

→ Also known as python interpreter

Byte Code is **Not** machine Code

→ Python specific interpretation

→ CPython, Jython, IronPython, Stackless, PyPy
 ↑
 standard implementation

Machine code == direct instruction to the python

Mutable and Immutable

In Python, everything is an object.

Immutable == The reference which is made inside the memory can never be changed.

Eg.

```
>>> x = 10
```

```
>>> y = x
```

Memory diagram:

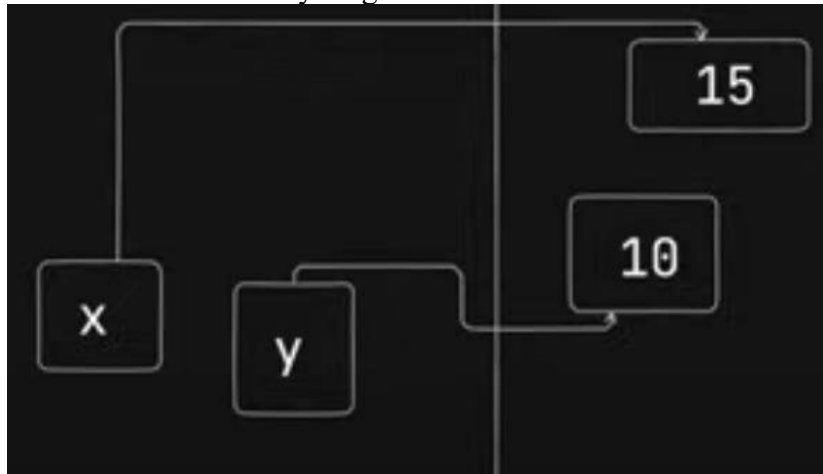


Now change in the code.

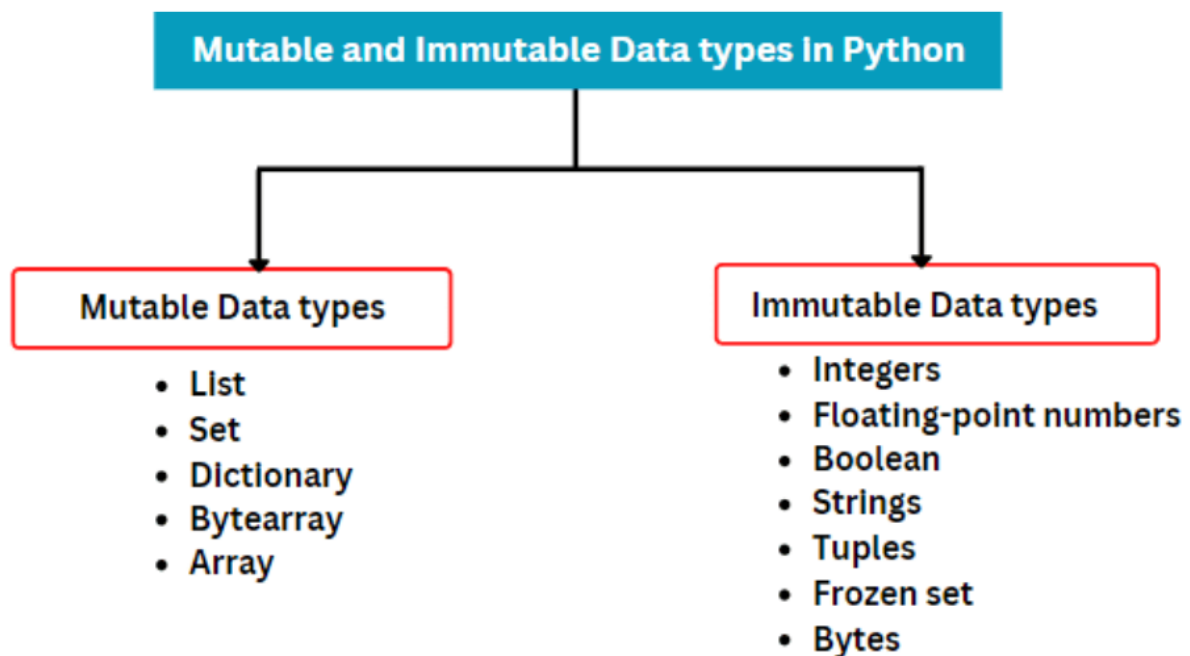
```
>>> x = 15
```

What will be y =?

Let's see in the memory diagram:



Here we can clearly see, the reference of x is change to 15 but y is still referencing to 10.
So, y=10;
And hence, it is immutable.



DATA TYPES / OBJECT TYPES:

- Number: 1234, 3.14, 3+4j, ob111, Decimal(), Fraction()
- String : 'spam', "Bob's", b'a\x01c', u'sp\xc4m'
- List : [1, 'three', [2, 'kumar'], 4.5] : indexing start from 0. list(range(10)): method to create list.

```
>>> mylist= [1,2,3,['three', 4, 5, 3.4]]
>>> mylist[2]
3
>>> mylist[3]
['three', 4, 5, 3.4]
>>> mylist[0]
```

- List : [1, 'three', [2, 'kumar'], 4.5] : indexing start from 0. list(range(10)): method to create list.
- Tuple : ('food', 1, 4, U), tuple('spam'), namedtuple
- Dictionary : {'food' : 'Span', 'taste' : 'yum'}, We give indexing for this. It has key-value pair, dict(hours=10): method to create a dictionary.
- Set : set('abc'), {'a', 'b', 'c'} : stores unique value.
- File : open('eggs.txt'), open(r'C:\ham,bin', 'wb') -- not a datatype but a mechanism to handle files. Python is capable of handling multiple file type.
- Boolean : True, False
- None : None -- empty references. eg, if u query for a temperature from an API but if does not respond, so in this case this empty references is used.
- Functions, modules, classes
- Advance: Decorators, Generators, Iterators
- MetaProgramming

Basic overview of each:

```
>>> 10 + 10
20
>>> 2*10
20
>>> 2**10
1024
>>> 2**100
1267650600228229401496703205376
>>> 2*3.14
6.28
>>> import math
>>> math.pi
3.141592653589793
>>> import random
>>> random.random()
0.27021201820854324
>>> random.random()
0.8876339548278619
>>> random.choice([1,2,3,4,5])
3
>>> random.choice([1,2,3,4,5])
3
>>> random.choice([1,2,3,4,5])
2
>>> username = "chaiaurcode"
>>> len(username)
11
>>> username[0]
'c'
username[0] = 'A'
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: 'str' object does not support item assignment

When we tried to replace zeroth index of username with new value, error is given because “String” is an immutable object.

```
>>> dir(username)
['_add_', '_class_', '_class_getitem_', '_contains_', '_delattr_', '_delitem_', '_dir_', '_doc_', '_eq_', '_format_', '_ge_', '_getattr_', '_ibute_', '_getitem_', '_getstate_', '_gt_', '_hash_', '_iadd_', '_imul_', '_init_', '_init_subclass_', '_iter_', '_le_', '_len_', '_lt_', '_mul_', '_ne_', '_new_', '_reduce_', '_reduce_ex_', '_repr_', '_reversed_', '_rmul_', '_setattr_', '_setitem_', '_sizeof_', '_str_', '_subclasshook_', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
```

dir (username): list out all the methods available in python for the object/ datatype username.

```
>>> mylist = [123, "chai", 3.14]
>>> mylist
[123, 'chai', 3.14]
>>> len(mylist)
3
>>> mylist[0]
123
>>> myD = {'one':'lemon', 'two':'ginger', 'comic':}
>>> myD['comic']
'naagraj'
>>> myD['comics']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'comics'
```

- mylist is same as array in other language, it has same properties as array.
- In dictionary, when key is not available, it gives key error.

```
>>> myTup = (1,2,4)
>>> myTup[0]
1
>>> myTup[4]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: tuple index out of range
```

INTERNAL WORKING OF PYTHON:

****We never assign any datatype to variable in python, it is referenced only in memory.**

a = 3

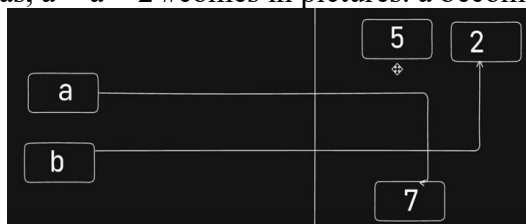
here a is not of datatype "Number". 3 is of 'Number' type which is stored in a variable a.

```
>>> a = 5
>>> b = 2
>>> a
5
>>> b
2
>>> a = a + 2
>>> a
7
>>>
```

Let's understand it through memory referencing.
initially, this happens.



as, a = a + 2 #comes in pictures. a becomes 7. Now, reference of a is changed to 7.



NUMBERS IN PYTHON

number carry everything in python – int, float, even sets are internally equal to number.
boolean is also number internally.

```
>>> x = 2
>>> y = 3
>>> z = 4
>>> x+y
5
>>> x + (y * z)
14
>>> 50 + 22.66
72.66
>>> float(50) + 22.66
72.66
```

try using same datatypes explicitly for better code readability.

```
>>> z ** 2
16
>>> z ** 10
1048576
>>> 2 ** 100
1267650600228229401496703205376
>>> 2 ** 1000
10715086071862673209484250490600018105614048117055336074437503883703510511249361224931983788156958581275946729
17553146825187145285692314043598457757469857480393456777482423098542107460506237114187795418215304647498358194
1267398767559165543946077062914571196477686542167660429831652624386837205668069376
```

Python has highest no handling capacity and high precision.

**Python has no precision as infinite but it lacks in decimal values.

```
>>> True is 1
<stdin>:1: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
False
>>> True + 4
5
>>> 2.2 + 2.2 + 2.2
6.6000000000000005
>>> (2.2 + 2.2 + 2.2 ) - 6.6
8.881784197001252e-16
```

** for handling decimal error in python, we need to import decimal library.

```
>>> from decimal import Decimal
>>> Decimal('1.1') + Decimal('1.1') + Decimal('1.1')
Decimal('3.3')
```

Syntactically true:

```
>>> 2<3<4
True
```

But production level, we must write it as more readable:

```
>>> 2<3 and 3<4
True
```

Difference between `__repr__` and `__str__`:

Points	<code>str()</code>	<code>repr()</code>
Return Value	Returns a human-readable string representation of the object	Returns an unambiguous string representation of the object
Usage	Used for creating user-friendly output and for displaying the object as a string	Used for debugging and development purposes to get the complete information of an object
Examples	<code>str(123)</code> returns <code>'123'</code>	<code>repr(123)</code> returns <code>'123'</code>
	<code>str('hello')</code> returns <code>'hello'</code>	<code>repr('hello')</code> returns <code>"'hello'"</code>
	<code>str([1, 2, 3])</code> returns <code>'[1, 2, 3]'</code>	<code>repr([1, 2, 3])</code> returns <code>'[1, 2, 3]'</code>
	<code>str({'name': 'John', 'age': 30})</code> returns <code>"{'name': 'John', 'age': 30}"</code>	<code>repr({'name': 'John', 'age': 30})</code> returns <code>"{'name': 'John', 'age': 30}"</code>

import math:

```
>>> import math
>>> math.floor(3.4)
3
>>> math.floor(-3.4)
-4
```

floor: it gives closest no below the value.

math.trunc(): this goes towards zero in no line:

```
>>> math.trunc(3.4)
3
>>> math.trunc(-3.4)
-3
```

0o: represent octal literal :

```
>>> 0o20
16
```

0x: represent hexadecimal literal:

```
>>> 0xFF
255
```

0b: represent binary literal :

```
>>> 0b1000
8
```

*We can convert any no system into int with the help of integer method.
here is how:

```
>>> int('63', 8)
51
>>> int('1000', 8)
512
>>> int('1000', 2)
8
>>> int('100', 16)
256
>>> int('100', 18)
324
```

import random: this random library helps in generating random number between 0 to 1.
but it can used to perform various task:

```
>>> import random
>>> random.random()
0.8353640488848072
>>> random.random()
0.8020348595351968
>>> random.random()
0.7803471306242157
>>> random.randint(2,10)
3
>>> random.randint(2,10)
6
>>> l1 = ["chai", "masalachai", "ginger", "mint"]
>>> random.choice(l1)
'chai'
>>> random.choice(l1)
'chai'
>>> random.choice(l1)
'chai'
>>> random.choice(l1)
'mint'
```

SETS:

& represents: intersection

| represents: union

empty set is denoted as = set() because {}: it is dictionary in python.

```
>>> import random
>>> random.shuffle(l1)
>>> setone = {1,2,3,4}
>>> setone & {2,5}
{2}
>>> setone | {5,7,8}
{1, 2, 3, 4, 5, 7, 8}
>>> setone - {1, 2, 3, 4}
set()
>>> type({})
<class 'dict'>
```


Boolean

```
>>> True == 1
True
>>> False == 0
True
>>> True + 4
5
>>> True is 1
<stdin>:1: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
False
>>> type(True)
<class 'bool'>
```

Strings in Python:

```
>>> chai = "Lemon Chai"
>>> print(chai)
Lemon Chai
>>> first_char = chai[0]
>>> print(first_char)
L
>>> slice_chai = chai[0:6]
>>> print(slice_chai)
Lemon
>>> chai[-1]
'i'
>>> chai[0:]
'Lemon Chai'
>>> chai[:]
'Lemon Chai'
>>> chai[: ]
'Lemon Chai'
>>> print(chai.lower())
lemon chai
>>> print(chai.upper())
LEMON CHAI
>>> chai
'Lemon Chai'
```

```
>>> chai = "Lemon Chai"
>>> print(chai.replace("Lemon", "Ginger"))
Ginger Chai
>>> chai
'Lemon Chai'
```

- change a string into a list[]:

```
>>> chai = "Masala, Mint, Ice, Ginger"
>>> print(chai.split())
['Masala,', 'Mint,', 'Ice,', 'Ginger']
>>> print(chai.split(", "))
['Masala', 'Mint', 'Ice', 'Ginger']
```

split(): by default, it splits on the basis of spaces.

split(", "): this splits by removing ", " from the string.

- **Find something in string.**

```
>>> chai = "Masala Chai"
>>> print(chai.find("chai"))
-1
>>> print(chai.find("Chai"))
7
```

If nothing is found in the string. It returns -1.

If found, it gives the starting index of the character, string.

- **Finding no of character or string**

```
>>> chai = "Masala Chai Chai Chai"
>>> print(chai.count("Chai"))
3
>>> print(chai.count("a"))
6
```

- **Importance of placeholder ({}).**

```
>>> chai_type = "Masala Chai"
>>> quantity = 2
>>> order = "I ordered {} cups of {} chai"
>>> order
'I ordered {} cups of {} chai'
>>> print(order.format(quantity, chai_type))
I ordered 2 cups of Masala Chai chai
```

- **Changing lists into string.**

```
>>> chai_variety = ["Lemon", "Masala", "Ginger"]
>>> print(chai_variety)
['Lemon', 'Masala', 'Ginger']
>>> print("".join(chai_variety))
LemonMasalaGinger
>>> print(" ".join(chai_variety))
Lemon Masala Ginger
>>> print("-".join(chai_variety))
Lemon-Masala-Ginger
>>> print(", ".join(chai_variety))
Lemon, Masala, Ginger
```

- **Length of string.**

```
>>> chai = "Masala Chai"
>>> print(len(chai))
11
```

- **Printing all the letters of a string.**

```
>>> chai = "Masala Chai"
>>> print(len(chai))
11
>>> for letter in chai:
...     print(letter)
...
M
a
s
a
l
a

C
h
a
i
```

- **Use of raw string in python : basically, used to print the addresses[path of directories] of windows.**

```
>>> chai = "c:\user\pwd"
File "<stdin>", line 1
    chai = "c:\user\pwd"
    ~~~~~
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 2-3: truncated \uXXXX escape

>>> chai = r"c:\user\pwd\"
File "<stdin>", line 1
    chai = r"c:\user\pwd\"
    ^
SyntaxError: unterminated string literal (detected at line 1)

>>> chai = r"c:\user\pwd"
>>> print(chai)
c:\user\pwd

>>> chai = "C:\\user\\pwd"
>>> chai
'C:\\user\\pwd'
>>> print(chai)
C:\user\pwd
```

Notes:

regarding errors: ‘\’ is a Unicode, so it can be included in the string as normal string. To include this, we either use “raw string” or “one extra ‘\’ ”. also, in raw string there should not be any ‘\’ in the end of string.

- **We can ask some question like this:**

```
>>> chai = "Masala Chai"
>>> print("Masala" in chai)
True
>>> print("Masalaa" in chai)
False
```

LIST [ARRAY] IN PYTHON

```
>>> tea_varieties = ["Black", "Green", "Oolong", "White"]
>>> print(tea_varieties)
['Black', 'Green', 'Oolong', 'White']
>>> print(tea_varieties[1:3])
['Green', 'Oolong']
>>> print(tea_varieties[1:])
['Green', 'Oolong', 'White']
>>> tea_varieties[3]= "Herbal"
>>> print(tea_varieties)
['Black', 'Green', 'Oolong', 'Herbal']
>>> tea_varieties
['Black', 'Green', 'Oolong', 'Herbal']
>>> tea_varieties[1:2]= "Lemon"
>>> tea_varieties
['Black', 'L', 'e', 'm', 'o', 'n', 'Oolong', 'Herbal']
```

when we are trying to insert or replace an element in list using slicing dicing then one-one character of element is inserted because each character is treated as an array. To solve this, we will pass element as list or array.

```
>>> tea_varieties = ["Black", "Green", "Oolong", "White"]
>>> tea_varieties[1:2]= ["Lemon"]
>>> tea_varieties
['Black', 'Lemon', 'Oolong', 'White']
```

```
>>> tea_varieties
['Black', 'Lemon', 'Oolong', 'White']
>>> tea_varieties[1:3]
['Lemon', 'Oolong']
>>> tea_varieties[1:3]=["Green", "Herbal"]
>>> tea_varieties
['Black', 'Green', 'Herbal', 'White']
```

```
>>> tea_varieties[1:1]= ["test", "test"]
>>> tea_varieties
['Black', 'test', 'test', 'Green', 'Herbal', 'White']
```

```
>>> tea_varieties
['Black', 'Mint', 'Masala', 'Green', 'Herbal', 'White']
>>> for tea in tea_varieties:
...     print(tea)
...
Black
Mint
Masala
Green
Herbal
White
```

- “\n” is the default after each iteration, we can define our own like this:

```
>>> for tea in tea_varieties:
...     print(tea, end="-")
...
Black-Mint-Masala-Green-Herbal-White->>>
```

- we can ask question in this way:

```
>>> print(tea_varieties)
['Black', 'Mint', 'Masala', 'Green', 'Herbal', 'White']
>>> if "Oolong" in tea_varieties:
...     print("I have Oolong tea")
...
>>> []
```

nothing is printed as there was no “Oolong” tea.

- `append()` is used to add an element at the end of the list.

```
>>> tea_varieties.append("Oolong")
>>> tea_varieties
['Black', 'Mint', 'Masala', 'Green', 'Herbal', 'White', 'Oolong']
>>> if "Oolong" in tea_varieties:
...     print("I have Oolong tea")
...
I have Oolong tea
>>> print(tea_varieties)
['Black', 'Mint', 'Masala', 'Green', 'Herbal', 'White', 'Oolong']
```

- to remove last element, we use: `.pop()`

```
>>> tea_varieties
['Black', 'Mint', 'Masala', 'Green', 'Herbal', 'White', 'Oolong']
>>> tea_varieties.pop()
'Oolong'
>>> tea_varieties
['Black', 'Mint', 'Masala', 'Green', 'Herbal', 'White']
```

- to remove any specific element, we use `.remove("element")`

```
>>> tea_varieties
['Black', 'Mint', 'Masala', 'Green', 'Herbal', 'White']
>>> tea_varieties.remove("Green")
>>> tea_varieties
['Black', 'Mint', 'Masala', 'Herbal', 'White']
```

- to insert any element at a position

```
>>> tea_varieties
['Black', 'Masala', 'Herbal', 'White']
>>> tea_varieties.insert(1, "green")
>>> tea_varieties
['Black', 'green', 'Masala', 'Herbal', 'White']
```

- when we copy our list to another variable, our new variable get the same reference as previous one, but if we want new reference, we can use `.copy()` method:

```
>>> tea_varieties
['Black', 'green', 'Masala', 'Oolong', 'White']
>>> tea_varieties_copy = tea_varieties.copy()
```

see the difference clearly:

```
>>> tea_varieties_copy
['Black', 'green', 'Masala', 'Oolong', 'White']
>>> tea_varieties.append("Lemon")
>>> tea_varieties_copy
['Black', 'green', 'Masala', 'Oolong', 'White']
>>> tea_varieties
['Black', 'green', 'Masala', 'Oolong', 'White', 'Lemon']
```

if references would have been seen, there would have been change in both as well, see:

```
>>> tea_varieties
['Black', 'green', 'Masala', 'Oolong', 'White', 'Lemon']
>>> tea_varieties_copy=tea_varieties
>>> tea_varieties_copy.append("Mint")
>>> tea_varieties
['Black', 'green', 'Masala', 'Oolong', 'White', 'Lemon', 'Mint']
>>> tea_varieties_copy
['Black', 'green', 'Masala', 'Oolong', 'White', 'Lemon', 'Mint']
```

- list comprehension in list.

```
>>> squared_nos = [x**2 for x in range(10)]
>>> print(squared_nos)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> cube_no = [y**3 for y in range(10)]
>>> print(cube_no)
[0, 1, 8, 27, 64, 125, 216, 343, 512, 729]
```

DICTIONARY IN PYTHON

It stores elements in key-value pair. It is useful in various places, eg, if we want to get DOB out of all the elements. We can access DOB by DOB key.

```
>>> chai_types= {"Masala": "Spicy", "Ginger": "Zesty", "Green": "Mild" }
>>> chai_types
{'Masala': 'Spicy', 'Ginger': 'Zesty', 'Green': 'Mild'}
>>> chai_types["Masala"]
'Spicy'
>>> chai_types["Masalaaa"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'Masalaaa'
>>> chai_types.get("Masala")
'Spicy'
>>> chai_types.get("Masalaa")
```

dict is a special datatype so it comes with special method.

we can access the value of key by get("key"), however this doesn't throw any error if key is not available. But when we access any key normally, the error comes when key is not available.

- Change the value of any key

```
>>> chai_types
{'Masala': 'Spicy', 'Ginger': 'Zesty', 'Green': 'Mild'}
>>> chai_types["Green"]="Fresh"
>>> chai_types
{'Masala': 'Spicy', 'Ginger': 'Zesty', 'Green': 'Fresh'}
```

- print key-values using loop.

```
>>> chai_types
{'Masala': 'Spicy', 'Ginger': 'Zesty', 'Green': 'Fresh'}
>>> for chai in chai_types:
...     print(chai)
...
Masala
Ginger
Green
>>> for chai in chai_types:
...     print(chai, chai_types[chai])
...
Masala Spicy
Ginger Zesty
Green Fresh
```

- accessing key, value with loop and method: .items()

```
>>> for key, value in chai_types.items():
...     print(key,value)
...
Masala Spicy
Ginger Zesty
Green Fresh
```

- asking a question

```
>>> if "Masala" in chai_types:
...     print("I have masala chai")
...
I have masala chai
```

- len() method works well with dictionary.

```
>>> chai_types
{'Masala': 'Spicy', 'Ginger': 'Zesty', 'Green': 'Fresh'}
>>> print(len(chai_types))
3
```

- adding a key-value

```
>>> chai_types
{'Masala': 'Spicy', 'Ginger': 'Zesty', 'Green': 'Fresh'}
>>> chai_types["Earl Grey"]="Citrus"
>>> chai_types
{'Masala': 'Spicy', 'Ginger': 'Zesty', 'Green': 'Fresh', 'Earl Grey': 'Citrus'}
```

- pop() also works well, but we need to provide key here.

```
>>> chai_types
{'Masala': 'Spicy', 'Ginger': 'Zesty', 'Green': 'Fresh', 'Earl Grey': 'Citrus'}
>>> chai_types.pop("Ginger")
'Zesty'
>>> chai_types
{'Masala': 'Spicy', 'Green': 'Fresh', 'Earl Grey': 'Citrus'}
```

- by using .popitem(): we can remove the last added key-value in the dictionary. This does not require any key:

```
>>> chai_types
{'Masala': 'Spicy', 'Green': 'Fresh', 'Earl Grey': 'Citrus'}
>>> chai_types.popitem()
('Earl Grey', 'Citrus')
>>> chai_types
{'Masala': 'Spicy', 'Green': 'Fresh'}
```

- del method: it actually deletes the reference from the memory.

```
>>> chai_types
{'Masala': 'Spicy', 'Green': 'Fresh'}
>>> del chai_types["Green"]
>>> chai_types
{'Masala': 'Spicy'}
```

- dict inside a dict

```
>>> tea_shop = {
...     "chai": {"Masala": "Spicy", "Ginger": "Zesty"},
...     "Tea": {"Green": "Mild", "Black": "Strong"}
... }
>>> tea_shop
{'chai': {'Masala': 'Spicy', 'Ginger': 'Zesty'}, 'Tea': {'Green': 'Mild', 'Black': 'Strong'}}
```

```
>>> tea_shop
{'chai': {'Masala': 'Spicy', 'Ginger': 'Zesty'}, 'Tea': {'Green': 'Mild', 'Black': 'Strong'}}
>>> tea_shop["chai"]
{'Masala': 'Spicy', 'Ginger': 'Zesty'}
>>> tea_shop["Tea"]
{'Green': 'Mild', 'Black': 'Strong'}
>>> tea_shop["chai"]["Ginger"]
'Zesty'
```

```
>>> tea_shop["Tea"]["Black"]
'Strong'
```

- some more stuff

```
>>> squared_num = {x:x**2 for x in range(6)}
>>> squared_num
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
>>> squared_num.clear()
>>> squared_num
{}
```

- create a dict using list and string

```
>>> keys = ["Masala", "Ginger", "Lemon"]
>>> keys
['Masala', 'Ginger', 'Lemon']
>>> default_value = "Delicious"
>>> new_dict = dict.fromkeys(keys,default_value)
>>> new_dict
{'Masala': 'Delicious', 'Ginger': 'Delicious', 'Lemon': 'Delicious'}
>>> new_dict = dict.fromkeys(keys,keys)
>>> new_dict
{'Masala': ['Masala', 'Ginger', 'Lemon'], 'Ginger': ['Masala', 'Ginger', 'Lemon'], 'Lemon': ['Masala', 'Ginger', 'Lemon']}
```


TUPLES IN PYTHON:

Why do we need tuples, when we have list which does the same work?

Ans: List is mutable, Tuples is immutable.

```
>>> tea_types = ("Black", "Green", "Oolong")
>>> tea_types
('Black', 'Green', 'Oolong')
>>> tea_types[-1]
'Oolong'
>>> tea_types[0]
'Black'
>>> tea_types[0]="Lemon"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

```
>>> len(tea_types)
3
>>> more_tea = ("Herbal", "Earl Grey")
>>> all_tea = more_tea + tea_types
>>> all_tea
('Herbal', 'Earl Grey', 'Black', 'Green', 'Oolong')
>>> len(all_tea)
5
```

```
>>> all_tea
('Herbal', 'Earl Grey', 'Black', 'Green', 'Oolong')
>>> if "Herbal" in all_tea:
...     print("I have Herbal Tea")
...
I have Herbal Tea
```

```
>>> more_tea = ("Herbal", "Earl Grey", "Herbal")
>>> more_tea
('Herbal', 'Earl Grey', 'Herbal')
>>> more_tea.count("Herbal")
2
```

- tuples are used to unwrap tuples

```
>>> tea_types
('Black', 'Green', 'Oolong')
>>> (black, green, Oolong) = tea_types
>>> black
'Black'
>>> green
'Green'
>>> Oolong
'Oolong'
```

- type() find the type of the datatypes.

```
>>> type(black)
<class 'str'>
>>> type("Black")
<class 'str'>
>>> type(tea_types)
<class 'tuple'>
```

- **How to take an input?**

You can take input from terminal.

Or using `input("statement")`

The input taken from the input statement is always a str.

- **If- else conditional:**

If condition:

 print statement.

or

if condition:

 print statement

elif condition:

 print statement

else:

 print statement.

- **Writing single line if statement.**

`price = 12 if age >= 18 else 8.` => this means price is equal to 12 when age is greater or equal to 18, else 8.

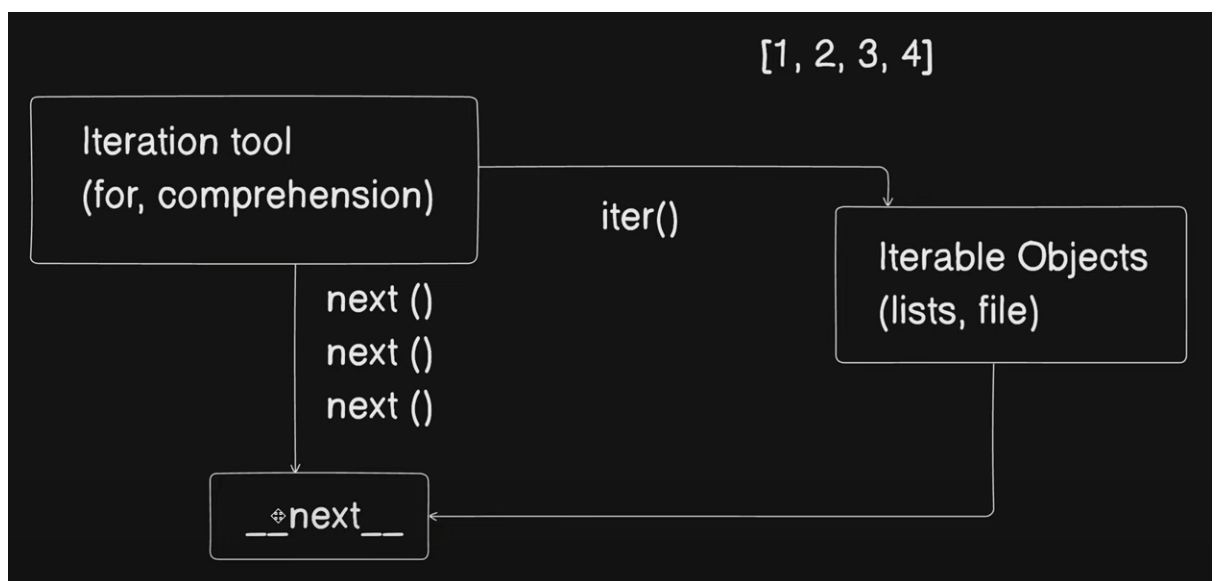
Loops:

`range(starting, ending):` ending is always exclusive.

`continue:` it skips the current iteration.

`break:` it exits all other existing operation. (say in a loop).

- Behind the scene of loop or iteration tool.



```
import time
print("chai is here")
username = "Mrityunjay"
print(username)
```

```
>>> f= open("chai.py")
>>> f.readline()
'import time\n'
>>> f.readline()
'print("chai is here")\n'
>>> f.readline()
'username = "Mrityunjay"\n'
>>> f.readline()
'print(username)'
>>> f.readline()
''
>>> f.readline()
''
```

when all the lines are ended in the file, then there is a '' (empty string) which is a kind of notation to say = file ended.

we can also use `__next__()` to read the files line by line, but in this case, when line will end, it will give error to stop iteration. we can also use loops to read the lines.

```
>>> f = open("chai.py")
>>> f.__next__()
'import time\n'
>>> f.__next__()
'print("chai is here")\n'
>>> f.__next__()
'username = "Mrityunjay"\n'
>>> f.__next__()
'print(username)'
>>> f.__next__()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

```
>>> for line in open("chai.py"):
...     print(line)
...
import time
print("chai is here")
username = "Mrityunjay"
print(username)
```

- to remove the space between the line, while printing through loop, we can modify our `print(line, end= '')`

```
>>> for line in open("chai.py"):
...     print(line, end='')
...
import time
print("chai is here")
username = "Mrityunjay"
print(username)>>> 
```

- printing the line using while loop

```
>>> f = open ('chai.py')
>>> while True:
...     line = f.readline()
...     if not line: break
...     print(line, end = '')
...
import time
print("chai is here")
username = "Mrityunjay"
print(username)>>>
```

- when we store a file inside a variable, that variable is also a iterable object.
iter(f) and f is same only for file.
Note: iter object is by default in files.

```
>>> f = open('chai.py')
>>> iter(f) is f
True
>>> iter(f) is f.__iter__()
True
```

- when we hold the reference of iter() of any list.
Note: memory reference of list iterator always points at starting.

```
>>> myList = [1, 2, 3, 4]
>>> I = iter(myList)
>>> I
<list_iterator object at 0x00000228F2C6B370>
>>> I.__next__()
1
>>> I
<list_iterator object at 0x00000228F2C6B370>
>>> I.__next__()
2
>>> I.__next__()
3
>>> I.__next__()
4
>>> I.__next__()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

- In case of list, only reference of variable is passed. Hence when a list is stored in a variable, it is not iterable.

```
>>> myNewList = [1, 2, 3]
>>> iter(myNewList) is myNewList
False
```

- Dictionary is also an iterable object.

```
>>> D = {'a': 1, 'b': 2}
>>> for key in D.keys():
...     print(key)
...
a
b
>>> I = iter(D)
>>> I
<dict_keyiterator object at 0x102f81990>
>>> next(I)
'a'
>>> next(I)
'b'
>>> next(I)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

- **Functions**

def square(parameter):
 print() or return

- **polymorphism in function**

polymorphism == ek naam, anek kaam.

- Function with default parameter:

```
def greet(name = "Mrityunjay"):
    return "Hello, " + name + "!"
print(greet())
print(greet("Hitesh"))
```

⇒ assign a default value to the variable in function parameter, if no argument is passed, then variable value will be treated as default parameter.

Output:

```
Hello, Mrityunjay!
Hello, Hitesh!
```

- Function with *args
args can take n no of parameter.

```
def sum_all(*args):
    return sum(args)
print(sum_all(1,11))
```

- **kwargs → it helps return output in key value pair.

```
def print_kwargs(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
print_kwargs(name= "Shaktiman", power = "lazer")
print_kwargs(name= "Shaktiman")
print_kwargs(name= "Shaktiman", power =
"lazer", enemy = "Dr. jackaal")
```

output:

```
name: Shaktiman
power: lazer
name: Shaktiman
name: Shaktiman
power: lazer
enemy: Dr. jackaal
```

- yield returns the value, but keep update of what is happening in the memory, also it manages the state.

```
def even_generator (limit):
    for i in range(2, limit+1,2):
        yield i

for num in even_generator(10):
    print(num)
```

output:

```
2
4
6
8
10
```

- **scope (namespace)**
in python, indentation is scope.
- **Closure (factory function)**

```
def chaicoder(num):
    def actual (x):
        return x ** num
    return actual
f = chaicoder(2)
g = chaicoder(3)
print (f) # only reference of f is passed here.
print(g) # only reference of g is passed here.
```

output:

```
<function chaicoder.<locals>.actual at 0x000001A72F989260>
<function chaicoder.<locals>.actual at 0x000001A72F989300>
```

let's execute the function here.

```
def chaicoder(num):
    def actual (x):
        return x ** num
    return actual
f = chaicoder(2)
g = chaicoder((3))
print (f(3))
print(g(3))
```

output:

```
9
27
```

• OBJECT ORIENTED PROGRAMMING.

⇒ Basic syntax:

```
class Car:
    def __init__(self,brand,model):
        self.brand = brand
        self.model = model
my_car = Car("Toyota", "Corolla")
print(my_car.brand, my_car.model)
```

output:

```
Toyota Corolla
```

- ⇒ self: sets the context. In lay man language, self helps to talk from Car and also this helps to refer itself or say (parameter or say attributes)
- ⇒ when you add __brand: then this attribute becomes private. This mean, class can access that attribute but object cannot.
- ⇒ static method → it is a method that can be accessed only by class, can not be accessed by instance (object) @staticmethod = also know as decorator

⇒ property decorator → it makes sure that, we can not modify any attribute.

```
@property
def model(self):
    return self.__model
```

also to call this method, we need only reference of the method model, not the execution

e.g.,

```
print(my_car.model)
```

- pass statement is a null operation in python.

```
class ElectricCarTwo(Battery, Engine, Car):
    pass
```

here pass does nothing, hence null operation.

- **Decorators**

decorators are like toll booth in real life.

⇒ a function is passed into a pipe (decorator)